

Hazardurile în *pipeline*-urile de instrucțiuni

3. Accelerarea prin transfer direct (*Forwarding*)

C. Metoda lui Tomasulo

Metoda lui Tomasulo se implementează pe o arhitectură superscalară (procesor cu multiple unități de execuție specializate, capabil să execute mai mult de o instrucțiune per ciclu) și se bazează pe câteva inovații structurale:

- stații de rezervare plasate pe intrările unităților de execuție (pentru preluarea operanzilor sursa specificați de fiecare instrucțiune).
- un bus de date comun (BDC) care asigură *forwarding*-ul; bus extins care, în fiecare ciclu, va transporta multiple rezultate produse de diferitele unități de execuție spre registrele destinație și respectiv stațiile de rezervare aflate în așteptarea acestora (*forwarding* în stațiile de rezervare pentru reducerea penalităților induse de hazardurile RAW).
- o schemă de recunoaștere simplă, bazată pe *tag*-uri asociate registrelor generale și respectiv stațiilor de rezervare
- biți *busy* asociați registrelor generale.

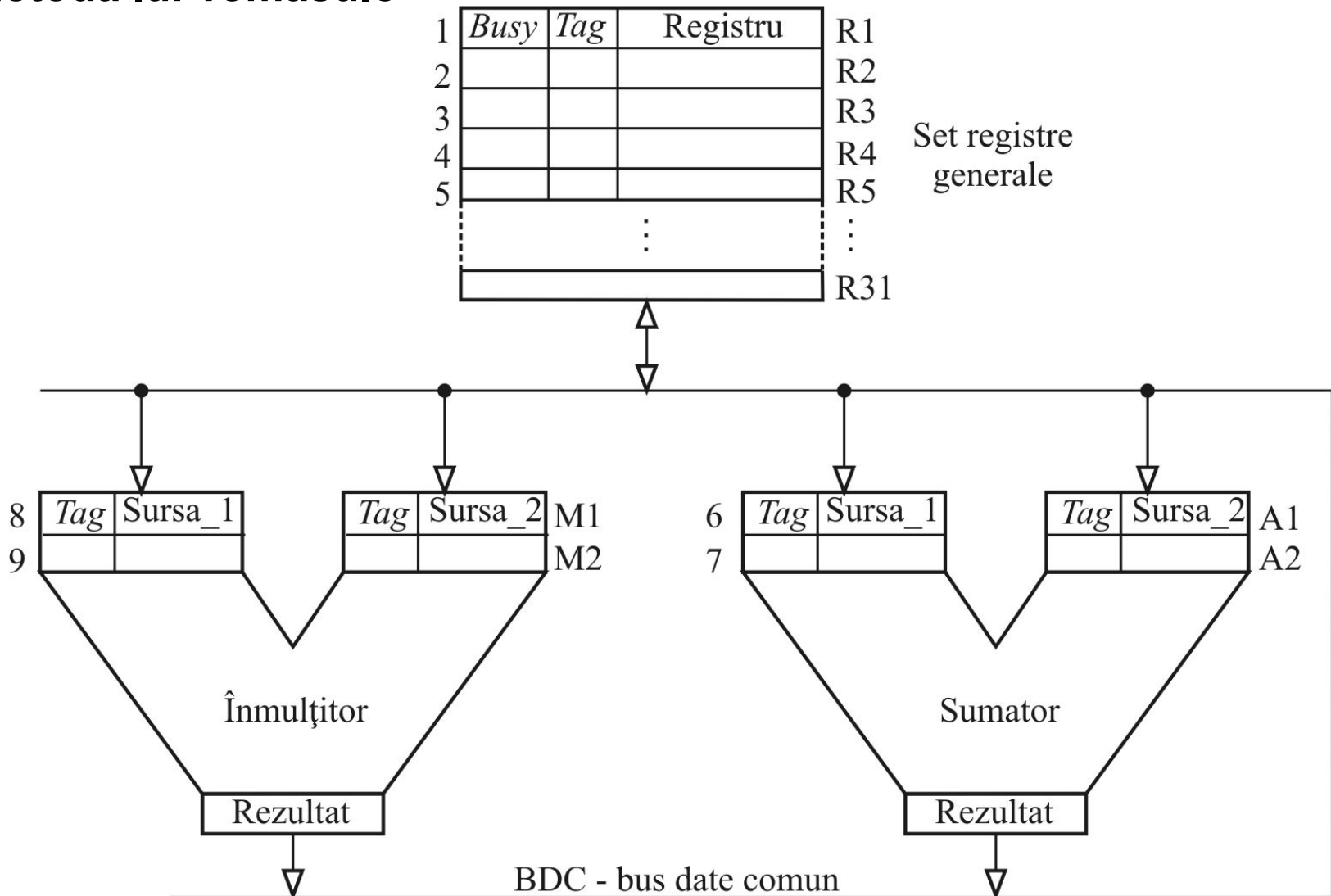
Vom considera următoarea secvență executată pe un procesor superscalar (arhitectură cu multiple unități de execuție):

```
i1:  ADD R2,R3,R4      ; R2 = R3 + R4
i2:  ADD R2,R2,R1      ; R2 = R2 + R1 (RAW și WAW pe R2)
```

Hazardurile în *pipeline*-urile de instrucțiuni

3. Accelerarea prin transfer direct (*Forwarding*)

C. Metoda lui Tomasulo



Arhitectură superscalară cu resurse dedicate implementării metodei lui Tomasulo

Hazardurile în *pipeline*-urile de instrucțiuni

3. Accelerarea prin transfer direct (*Forwarding*)

C. Metoda lui Tomasulo

După *fetch*, instrucțiunea **i1** intră în OF fiind alocată stației A1 pentru execuție:

- conținutul registrelor R3 și R4 este transmis registrelor sursa_1 și respectiv sursa_2 din stația de rezervare A1 (faza OF propriu-zisă).
- bitul *busy* aferent registrului R2 (registrul destinație) este setat la 1
- *tag*-ul registrului R2 este setat la valoarea 6 (**0110_B**) - *tag*-ul stației de adunare A1.

Pe ciclul *pipeline* următor, sumatorul (unitatea de adunare) startează execuția instrucțiunii **i1**. Pe același ciclu, se va executa faza OF aferentă instrucțiunii **i2** și i se va alocă stația A2 pentru execuție:

- conținutul registrului R1 este transmis în registrul sursa_2 din stația A2
- *tag*-ul registrului R2 (**0110_B**) va fi înscris în câmpul *tag* aferent registrului sursa_1 din stația A2 (nu pot fi preluate datele din R2 deoarece are bitul *busy* setat; hazard RAW)
- *tag*-ul lui R2 este setat la valoarea 7 (**0111_B**) - *tag*-ul stației de adunare A2
- sumatorul execută **i1** (preia din A1) și emite rezultatul (însoțit de *tag*-ul **0110_B**) pe BDC, de unde va fi încărcat în sursa_1 din stația A2 (egalitate la comparare *tag*-uri)

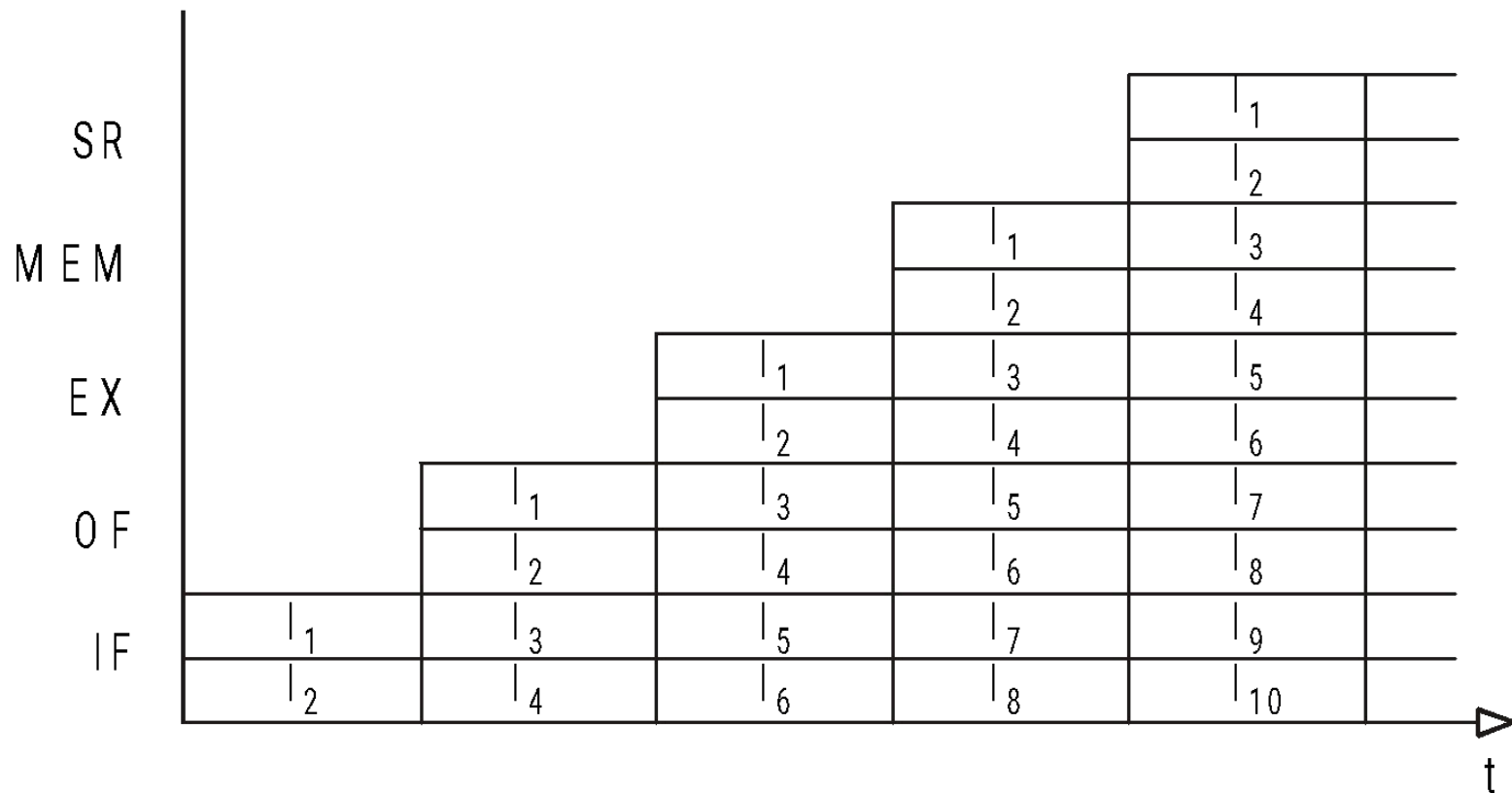
Pe ciclul *pipeline* următor, sumatorul execută instrucțiunea **i2** (preia din A2) și emite rezultatul (însoțit de *tag*-ul **0111_B**) pe BDC, de unde va fi încărcat în R2 (egalitate la comparare *tag*-uri).

Să observăm că rezultatul instrucțiunii **i1** nu mai ajunge efectiv în R2; este doar *forward*-at spre A2 ! Se obține maximum de eficiență pe RAW-uri !

Procesoare superscalare

1. Considerații generale

Un procesor superscalar este un procesor care execută concurrent mai multe instrucțiuni scalare. Altfel spus, procesarea superscalară se obține prin *fetch*-ul și respectiv execuția simultană a mai multor instrucțiuni. Prin urmare, un procesor superscalar necesită mai multe structuri *pipeline*; câte o structură pentru fiecare dintre instrucțiunile procesate concurrent.

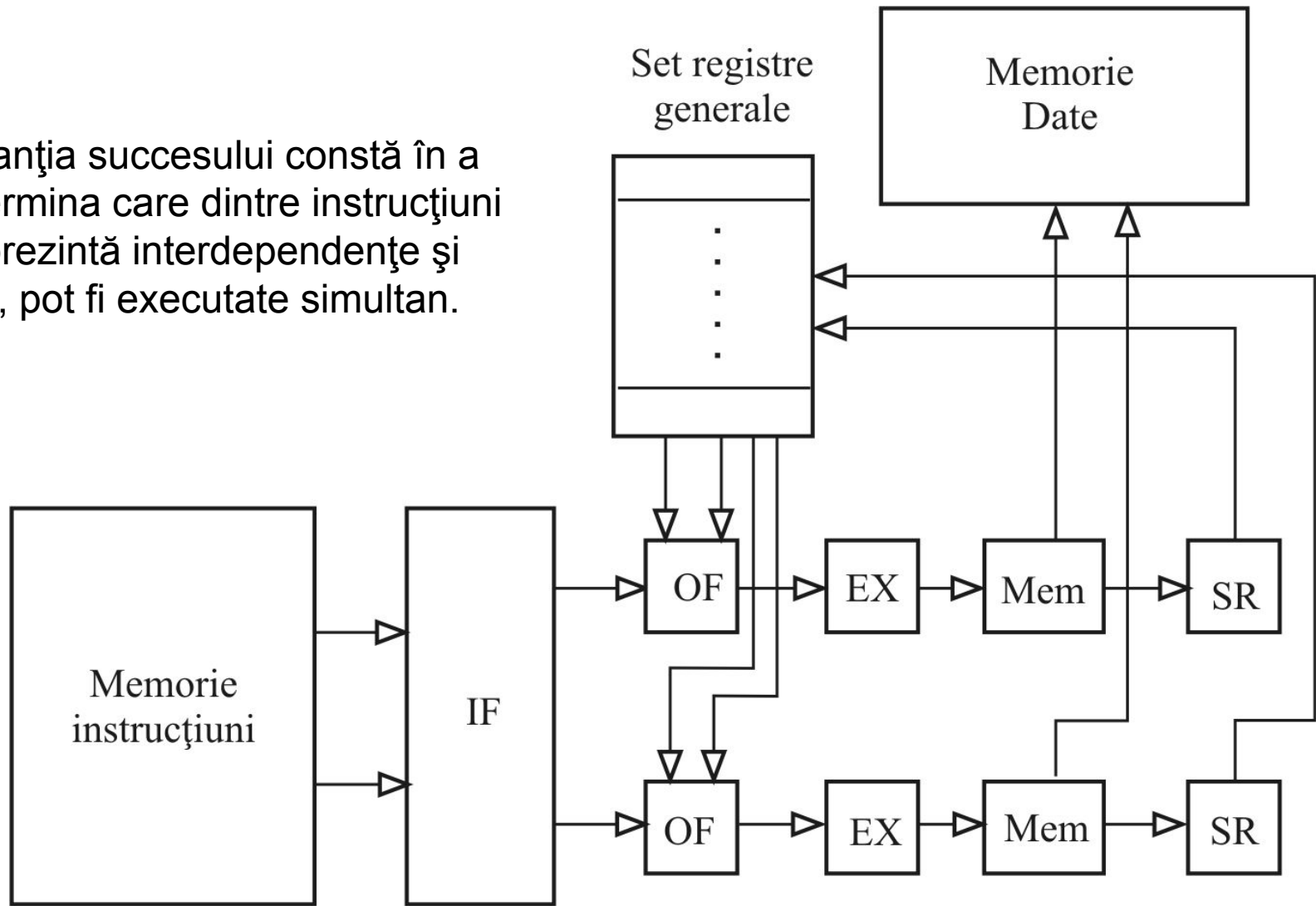


Procesarea instrucțiunilor într-un procesor superscalar cu două structuri *pipeline*

Procesoare superscalare

1. Considerații generale

Garanția succesului constă în a determina care dintre instrucțiuni nu prezintă interdependențe și deci, pot fi executate simultan.

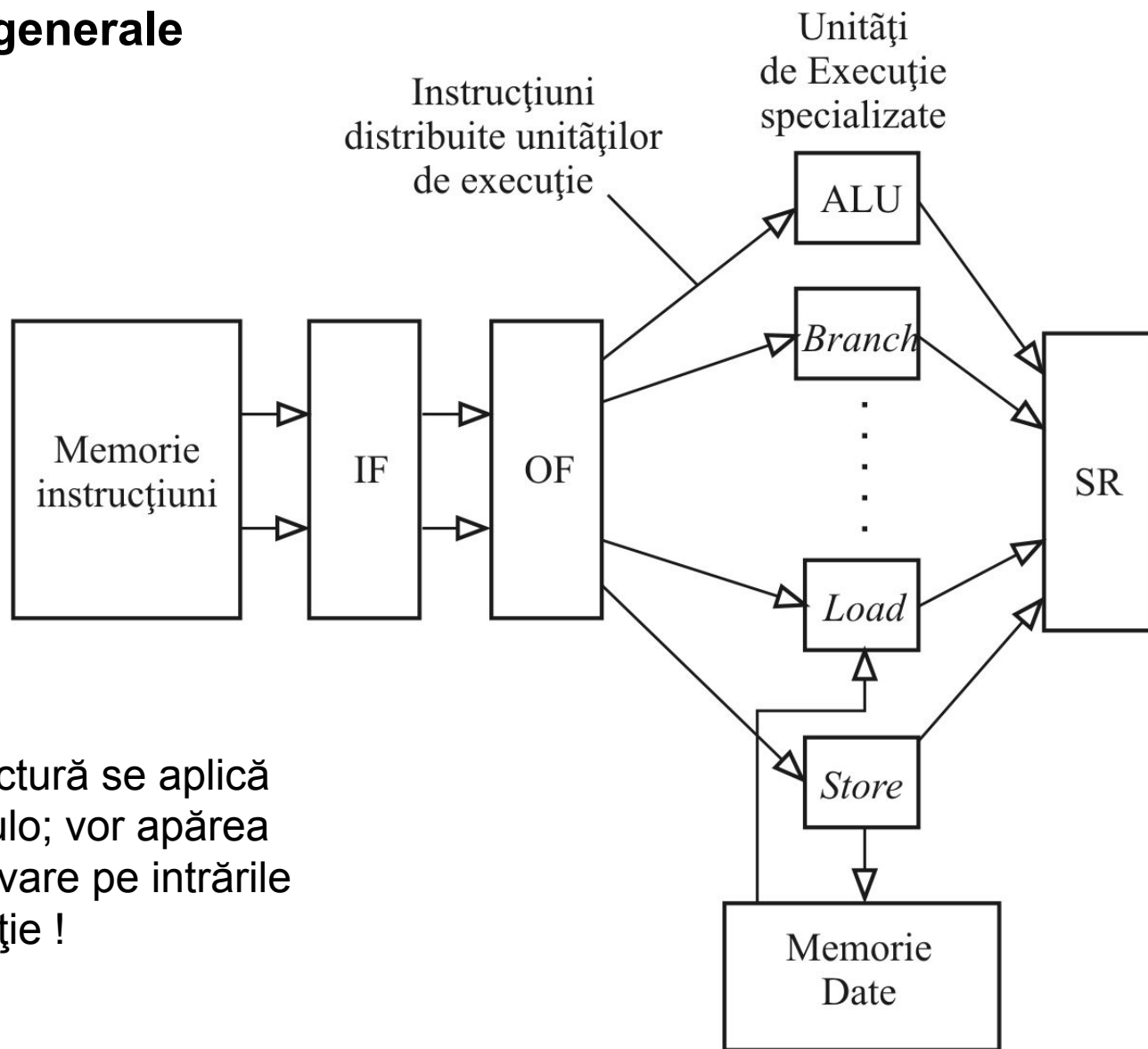


Procesor superscalar cu 2 structuri *pipeline*

(multiplicarea cascadelor *pipeline* o regăsim la primele procesoare superscalare)

Procesoare superscalare

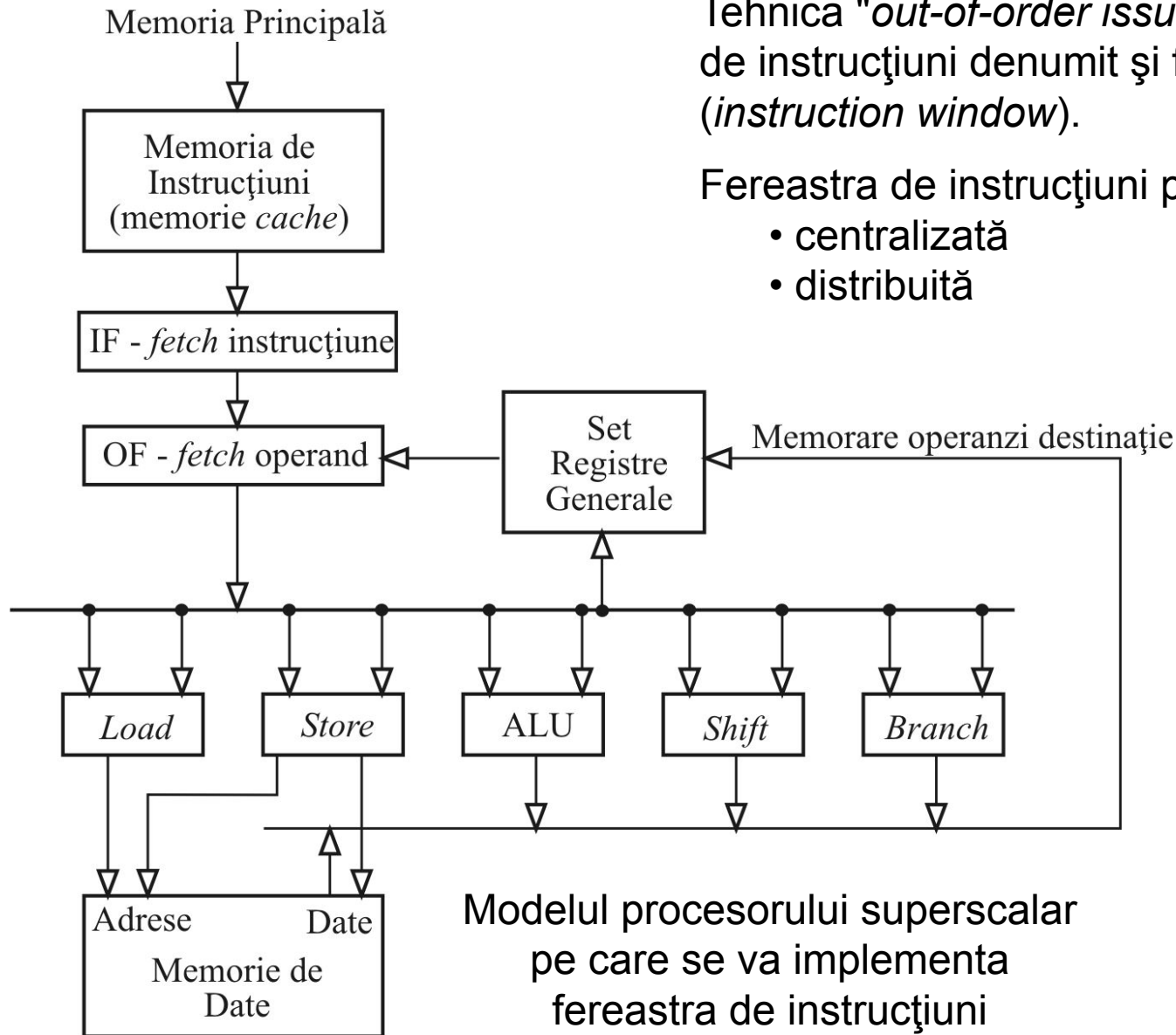
1. Considerații generale



Procesor superscalar cu unități de execuție specializate
(structură caracteristică celor mai noi procesoare superscalare)

Procesoare superscalare

2. Implementarea "*out-of-order issue*"



Tehnica "*out-of-order issue*" necesită un *buffer* de instrucțiuni denumit și fereastră de instrucțiuni (*instruction window*).

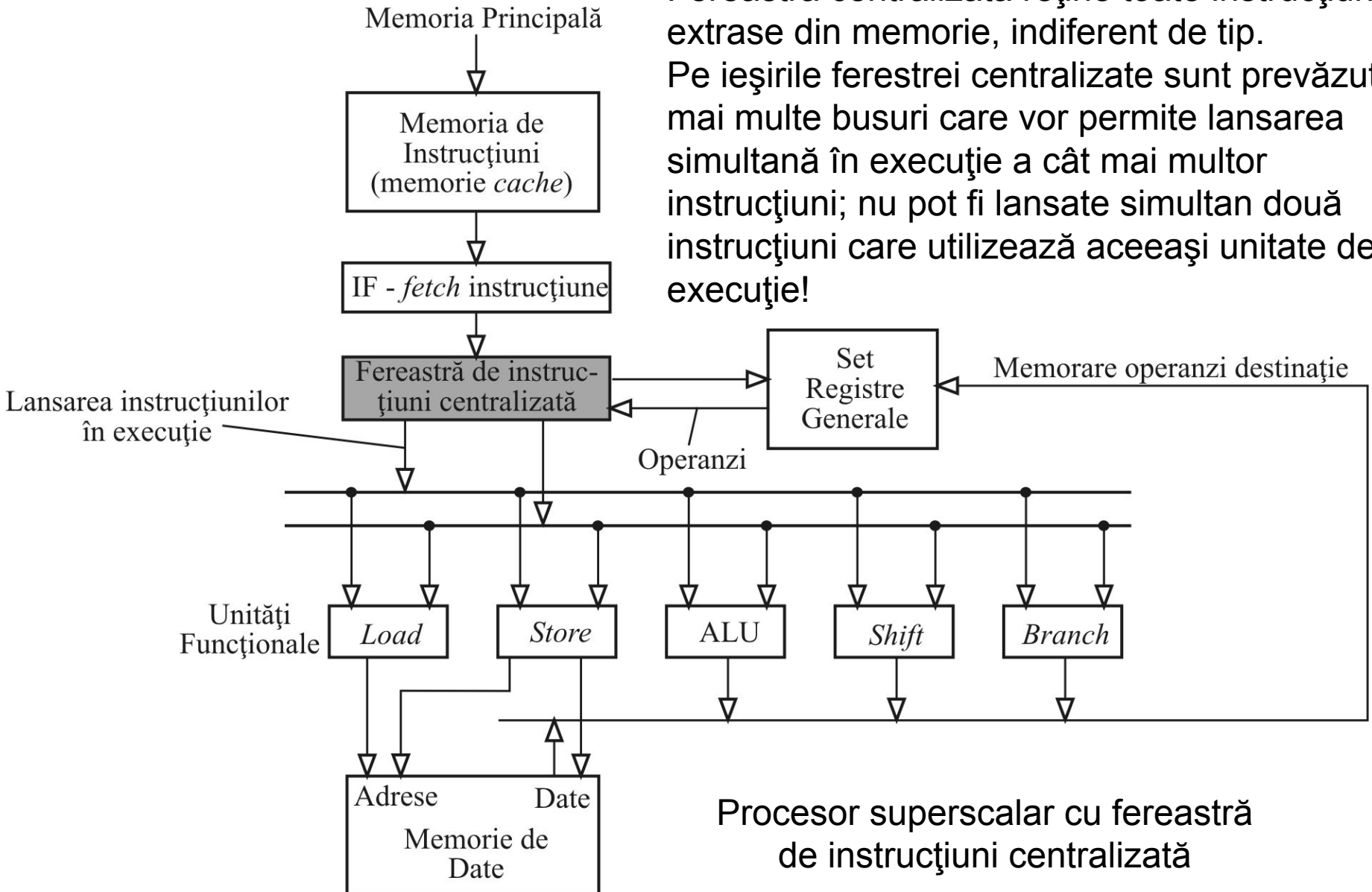
Fereastra de instrucțiuni poate fi:

- centralizată
- distribuită

Procesoare superscalare

2. Implementarea "*out-of-order issue*" / Fereastra centralizată

Fereastra centralizată reține toate instrucțiunile extrase din memorie, indiferent de tip. Pe ieșirile ferestrei centralizate sunt prevăzute mai multe busuri care vor permite lansarea simultană în execuție a cât mai multor instrucțiuni; nu pot fi lansate simultan două instrucțiuni care utilizează aceeași unitate de execuție!



Procesoare superscalare

2. Implementarea "*out-of-order issue*" / Fereastra centralizată

Fereastra de instrucțiuni va conține alături de *opcode*-ul instrucțiunii operanzii acesteia sau identificatorii registrelor sursă dacă operanzii sunt indisponibili (hazard RAW). O instrucțiune poate fi lansată în execuție către unitatea funcțională corespunzătoare numai dacă dispune de toți operanzii.

Când o instrucțiune trebuie să actualizeze registrul destinație (faza *Store Result*), fereastra de instrucțiuni este de asemenea accesată. Identificatorul registrului destinație va fi comparat cu toți identificatorii de registre sursă din fereastră. Toți identificatorii sursă pe care se obține egalitate la comparare vor fi înlocuiți cu respectivul rezultat (*forwarding* pe hazarduri RAW). Această actualizare a ferestrei se face simultan cu scrierea rezultatului în registrul destinație (faza *Store Result*).

Instrucțiune	Opcode	Registru Destinație	Operand 1	Registru Operand 1	Operand 2	Registru Operand 2
1	Operație 1	ID		ID	Valoare Operand	
2	Operație 2	ID		ID	Valoare Operand	
3	Operație 3	ID	Valoare Operand			ID
4	Operație 4	ID	Valoare Operand		Valoare Operand	
5	Operație 5	ID	Valoare Operand			ID
.	.	.	.	.	.	.
.	.	.	.	.	.	.
.	.	.	.	.	.	.

Conținutul ferestrei de instrucțiuni

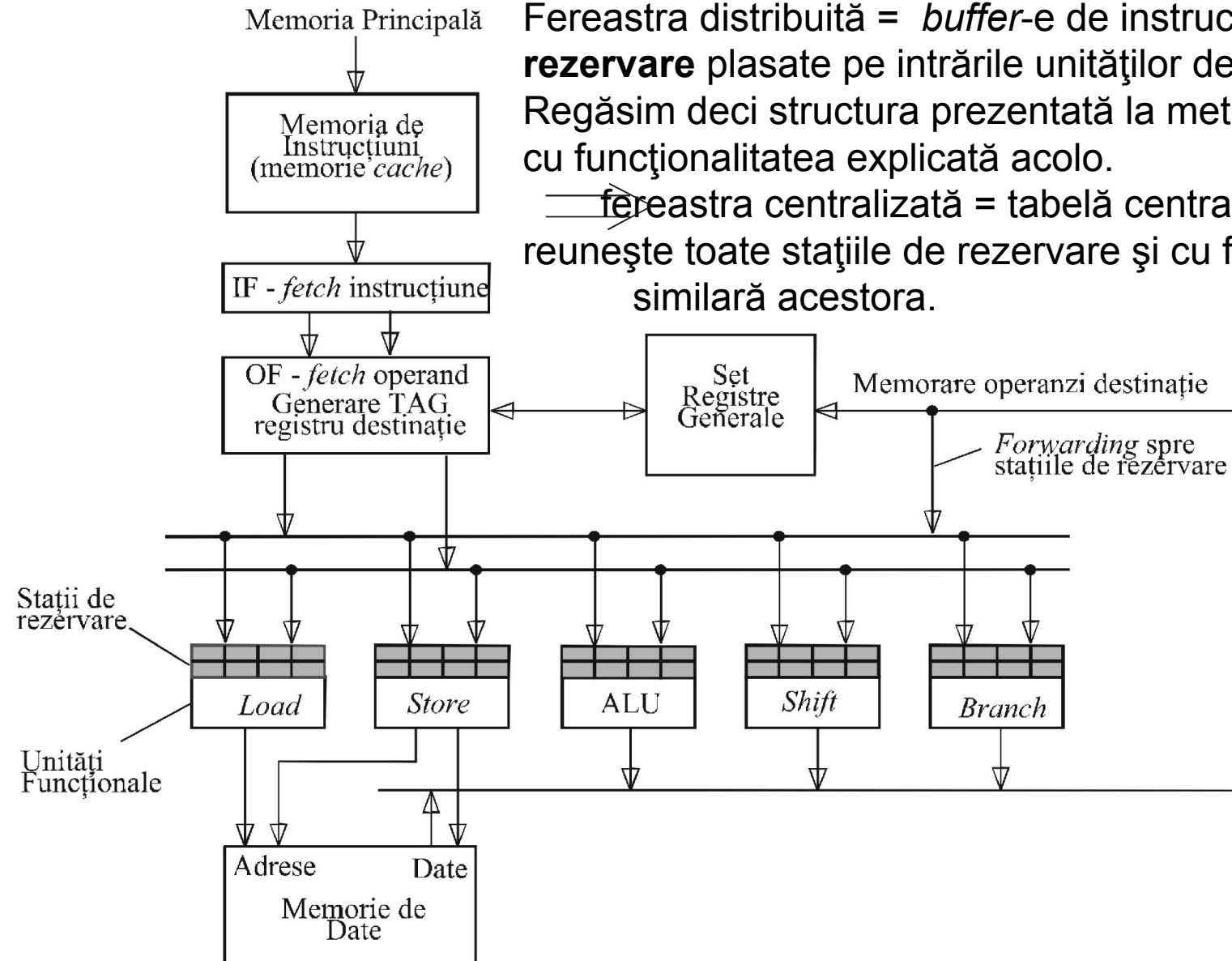
Procesoare superscalare

2. Implementarea "*out-of-order issue*" / Fereastra distribuită

Fereastra distribuită = *buffer*-e de instrucțiuni = **stații de rezervare** plasate pe intrările unităților de execuție.

Regăsim deci structura prezentată la metoda lui Tomasulo, cu funcționalitatea explicată acolo.

— fereastra centralizată = tabelă centralizată care reunește toate stațiile de rezervare și cu funcționalitate similară acestora.



Fereastra distribuită (implementată sub forma stațiilor de rezervare)

Procesoare superscalare

3. Redenumirea registrelor (*register renaming*)

A. Considerații generale

Tehnica redenumirii registrelor rezolvă hazardurile WAR și WAW și se implementează în *hardware*.

Fie secvența:

```
ADD R4,R1,R2    ; R4 = R1 + R2
ADD R1,R4,8      ; R1 = R4 + 8 (hazard WAR pe R1 și RAW pe R4)
ADD R4,R2,R3     ; R4 = R2 + R3 (hazarduri WAR și WAW pe R4)
```

Introducând două noi registre, R1a și R4a (*register renaming*), vom avea:

```
ADD R4,R1,R2    ; R4 = R1 + R2
ADD R1a,R4,8     ; R1a = R4 + 8
ADD R4a,R2,R3    ; R4a = R2 + R3
```

Prin redenumirea registrelor se elimină hazardurile WAR și WAW !

Noile registre sunt create la nevoie (instantaneu) și “distruse” în momentul în care nu mai există referințe nerezolvate la datele memorate în acestea.

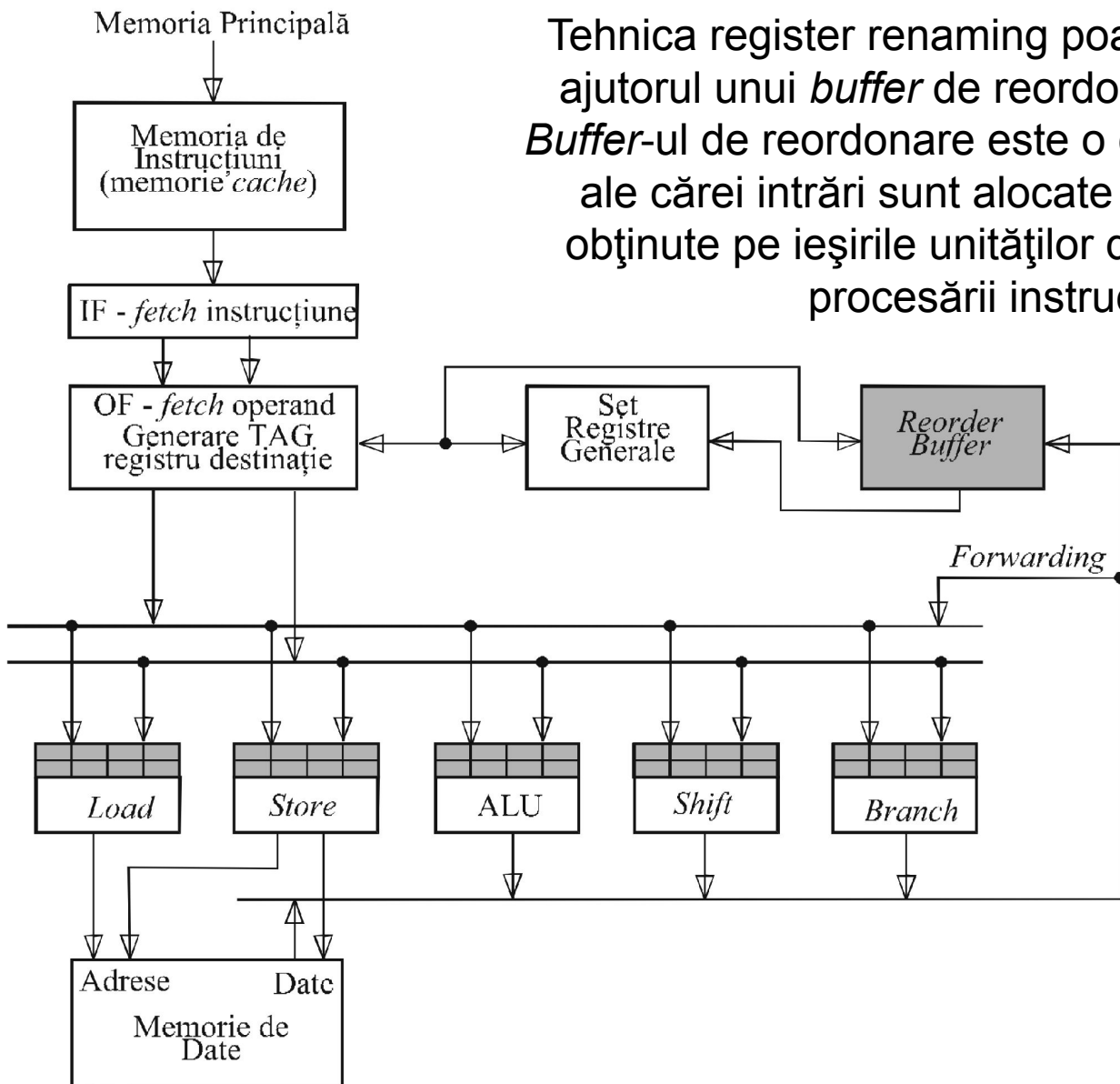
Tehnica *register renaming* nu rezolvă hazardurile RAW; pentru RAW-uri se aplică metoda lui Tomasulo !

Procesoare superscalare

3. Redenumirea registrelor (*register renaming*)

B. *Buffer*-ul de reordonare (*Reorder Buffer*)

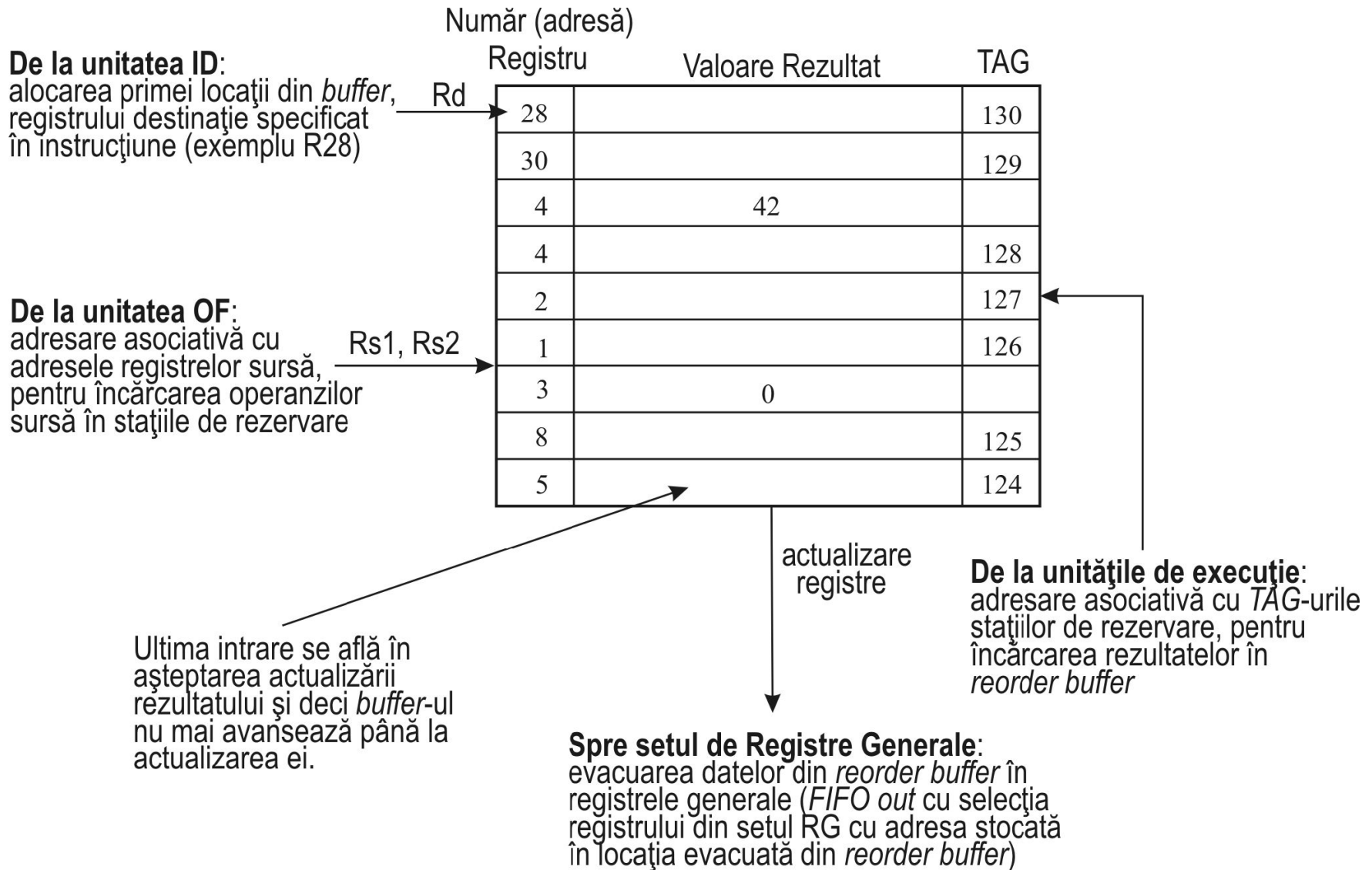
Tehnica register renaming poate fi implementată cu ajutorul unui *buffer* de reordonare (*reorder buffer*). *Buffer*-ul de reordonare este o coadă (memorie) FIFO ale cărei intrări sunt alocate dinamic rezultatelor obținute pe ieșirile unităților de execuție, în timpul procesării instrucțiunilor.



Procesoare superscalare

3. Redenumirea registrelor (*register renaming*)

B. *Buffer*-ul de reordonare (*Reorder Buffer*)

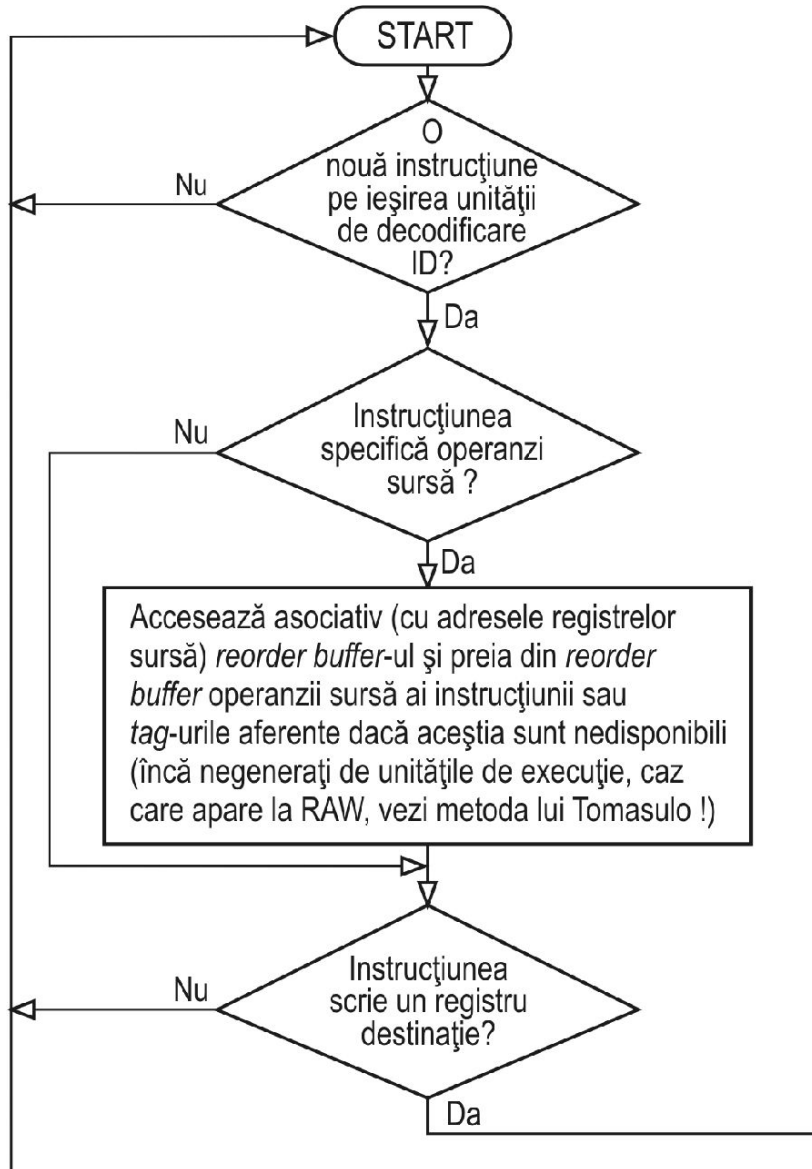


Organizarea *buffer*-ului de reordonare

Procesoare superscalare

3. Redenumirea registrelor (*register renaming*)

B. *Buffer-ul de reordonare (Reorder Buffer)*



Algoritmul de funcționare aferent *reorder buffer*-ului

